Please solve all these questions given in the attached PDF

I'll solve all the Divide-and-Conquer and Computability Theory problems from your PDF with detailed, understandable explanations.[1]

Computability Theory

Decidability and Recognizability

Problem 1: Prove that $A_{DFA} = \{L | M(L) \text{ accepts } L\}$ is decidable

Clarification: I believe this should be $A_{DFA} = \{\langle D, w \rangle | D \text{ is a DFA and } D \text{ accepts } w\}$ [1].

Proof: To decide A_{DFA}:

```
Algorithm:
1. Input: <D, w> where D is a DFA and w is a string
2. Simulate D on input w
3. If D accepts w, ACCEPT
4. If D rejects w, REJECT
```

Why this decides A_{DFA}: DFAs always halt (they process exactly $|w|$ steps). The simulation runs in finite time $O(|w|)$. Since we get a definite answer (accept or reject) in finite time, A_{DFA} is decidable [1].

Problem 2: Is $A_{TM} = \{\langle M, w \rangle | M \text{ accepts } w\}$ decidable? Is it recognizable?

Decidability: NO, A_{TM} is NOT decidable.[1]

Proof (by contradiction): Assume A_{TM} is decidable via decider H. Construct machine D:

```
D(<M>):
   Run H on <M, <M>>
   If H accepts, REJECT
   If H rejects, ACCEPT
```

Now run D on itself: $D(\langle D \rangle)$

- If D accepts $\langle D \rangle$, then by definition, H rejects $\langle D, \langle D \rangle \rangle$, meaning D doesn't accept $\langle D \rangle$. Contradiction.
- If D rejects $\langle D \rangle$, then H accepts $\langle D, \langle D \rangle \rangle$, meaning D accepts $\langle D \rangle$. Contradiction.[1]

Recognizability: YES, A_{TM} IS recognizable.[1]

Proof: Universal Turing Machine U recognizes A_{TM}:

```
U(<M, w>):
   Simulate M on w
   If M accepts, ACCEPT
   If M rejects, REJECT
   If M loops, U also loops
```

U accepts exactly when M accepts w, so A_{TM} is recognizable (but may not halt on inputs not in the language).[1]

Problem 3: If both L and $\overline{L}$ are recognizable, prove that L is decidable

Proof: Let M_1 recognize L and M_2 recognize $\overline{L}$.[1]

Decider D for L:

```
D(w):
   Run M_1 and M_2 in parallel (dovetailing)
   If M_1 accepts, ACCEPT
   If M_2 accepts, REJECT
```

Correctness:

- If $w \in L$, then M_1 accepts eventually. D accepts.
- If $w \notin L$, then $w \in \overline{L}$, so M_2 accepts eventually. D rejects.
- One of them must accept (since $L \cup \overline{L} = \Sigma^*$), so D always halts.[1]

Therefore, L is decidable.[1]

Problem 4: Is $L = \{\langle M \rangle | M \text{ accepts the empty string } \epsilon\}$ decidable? Is it recognizable?

Decidability: NO.[1]

Proof: This is a variant of A_{TM}. We can reduce A_{TM} to this problem. Given $\langle M, w \rangle$, construct M':

```
M'(x):
   Ignore input x
   Run M on w
   If M accepts w, ACCEPT
```

Then $M \text{ accepts } w \iff M' \text{ accepts } \epsilon$. Since A_{TM} is undecidable and reduces to L, L is undecidable.[1]

Recognizability: YES.[1]

Proof: Recognizer:

```
R(<M>):
   Simulate M on ε
   If M accepts, ACCEPT
   Otherwise, loop or reject
```

This recognizes L.[1]

Problem 5: Prove that $L = \{\langle M \rangle | M \text{ accepts } \langle M \rangle\}$ is Turing-recognizable, but $\overline{L}$ is not Turing-recognizable

Part 1: L is recognizable

Proof: Universal TM recognizes L:

```
U(<M>):
   Simulate M on <M>
   If M accepts, ACCEPT
   Otherwise, loop
```

L is recognizable.[1]

Part 2: $\overline{L}$ is NOT recognizable

Proof (by contradiction): Assume $\overline{L}$ is recognizable. Then by Problem 3, since both L and $\overline{L}$ would be recognizable, L would be decidable.[1]

But we can show L is undecidable using diagonalization (similar to A_{TM}): if L were decidable via decider H, construct:

```
D(<M>):
   Run H on <M>
   If H accepts (M accepts <M>), REJECT
   If H rejects, ACCEPT
```

Running D on itself gives contradiction. Therefore, L is not decidable, so $\overline{L}$ cannot be recognizable.[1]

Reducibility

Problem 1: Reduce A_{TM} to $X = \{\langle M \rangle | L(M) \text{ is finite}\}$ to show X is undecidable

Reduction: Given $\langle M, w \rangle$ (instance of A_{TM}), construct machine M':

```
M'(x):
   If x has the form 1^n for some n:
      Simulate M on w
      If M accepts w, ACCEPT x
   REJECT
```

Correctness:

- If M accepts w: M' accepts all strings of form 1^n, so $L(M')$ is infinite
- If M doesn't accept w: M' accepts nothing, so $L(M')$ is finite (empty)[1]

Therefore: $M \text{ accepts } w \iff L(M') \text{ is infinite} \iff M' \notin X$

This is a reduction from A_{TM} to $\overline{X}$, showing X is undecidable.[1]

Problem 2: Reduce A_{TM} to PAL = $\{\langle M \rangle | M \text{ accepts some palindrome}\}$ to show PAL is undecidable

Reduction: Given $\langle M, w \rangle$, construct M':

```
M'(x):
   If x is a palindrome:
      Simulate M on w
      If M accepts w, ACCEPT
   REJECT
```

Correctness:

- If M accepts w: M' accepts all palindromes, so $M' \in \text{PAL}$
- If M doesn't accept w: M' accepts no palindromes, so $M' \notin \text{PAL}$[1]

Therefore: $M \text{ accepts } w \iff M' \in \text{PAL}$

This is a reduction from A_{TM} to PAL, showing PAL is undecidable.[1]

Problem 3: Reduce A_{TM} to $L_{1000} = \{\langle M \rangle | M \text{ accepts a string of length } 1000\}$

Reduction: Given $\langle M, w \rangle$, construct M':

```
M'(x):
   If |x| = 1000:
      Simulate M on w
      If M accepts w, ACCEPT
   REJECT
```

Correctness:

- If M accepts w: M' accepts all strings of length 1000, so $M' \in L_{1000}$
- If M doesn't accept w: M' accepts no string of length 1000, so $M' \notin L_{1000}$[1]

Therefore: $M \text{ accepts } w \iff M' \in L_{1000}$

This is a reduction from A_{TM} to L_{1000}, showing L_{1000} is undecidable.[1]

Divide-and-Conquer Problems

Problem 1: Divide-and-Conquer Integer Multiplication

Multiply: 10011011 (155 in decimal) × 10111010 (186 in decimal)[1]

Using Karatsuba Algorithm:

Both numbers are 8 bits. Split into 4-bit halves:

- $X = 10011011 = 1001 \cdot 2^4 + 1011$
- $Y = 10111010 = 1011 \cdot 2^4 + 1010$

Let $a = 1001_2 = 9$, $b = 1011_2 = 11$, $c = 1011_2 = 11$, $d = 1010_2 = 10$

Karatsuba formulas:

1. $ac = 9 \times 11 = 99 = 1100011_2$
2. $bd = 11 \times 10 = 110 = 1101110_2$
3. $(a+b)(c+d) = (9+11)(11+10) = 20 \times 21 = 420$
4. $(a+b)(c+d) - ac - bd = 420 - 99 - 110 = 211$

Result:
$X \times Y = ac \cdot 2^8 + [(a+b)(c+d) - ac - bd] \cdot 2^4 + bd$
$= 99 \cdot 256 + 211 \cdot 16 + 110$
$= 25344 + 3376 + 110 = 28830$

In binary: $28830_{10} = 111000010011110_2$[1]

Verification: $155 \times 186 = 28830$ ✓

Problem 2: Comparing Three Algorithms

Algorithm A: $T(n) = 5T(n/2) + O(n)$[1]

Using Master Theorem: $a = 5$, $b = 2$, $f(n) = n$

- $n^{\log_b a} = n^{\log_2 5} \approx n^{2.32}$
- Since $f(n) = n = O(n^{2.32-\epsilon})$ for some $\epsilon > 0$

Case 1 applies: $T(n) = \Theta(n^{\log_2 5}) = \Theta(n^{2.32})$[1]

Algorithm B: $T(n) = 2T(n-1) + O(1)$[1]

Expanding:
$T(n) = 2T(n-1) + c = 2(2T(n-2) + c) + c = 2^2T(n-2) + 2c + c$
$= 2^kT(n-k) + c(2^{k-1} + 2^{k-2} + \ldots + 1) = 2^kT(n-k) + c(2^k - 1)$

When $k = n$: $T(n) = 2^nT(0) + c(2^n - 1) = \Theta(2^n)$[1]

Algorithm C: $T(n) = 9T(n/3) + O(n^2)$[1]

Using Master Theorem: $a = 9$, $b = 3$, $f(n) = n^2$

- $n^{\log_b a} = n^{\log_3 9} = n^2$
- Since $f(n) = n^2 = \Theta(n^2)$

Case 2 applies: $T(n) = \Theta(n^2 \log n)$ [1]

Comparison:

- Algorithm A: $O(n^{2.32})$
- Algorithm B: $O(2^n)$
- Algorithm C: $O(n^2 \log n)$

Best choice: Algorithm C [1]

Problem 3: Solving Recurrence Relations

1. $T(n) = 2T(n/3) + 1$

Master Theorem: $a = 2$, $b = 3$, $f(n) = 1$

- $n^{\log_3 2} \approx n^{0.63}$
- $f(n) = 1 = O(n^{0.63-\epsilon})$

Case 1: $T(n) = \Theta(n^{\log_3 2})$ [1]

2. $T(n) = 5T(n/4) + n$

Master Theorem: $a = 5$, $b = 4$, $f(n) = n$

- $n^{\log_4 5} \approx n^{1.16}$
- $f(n) = n = O(n^{1.16-\epsilon})$

Case 1: $T(n) = \Theta(n^{\log_4 5})$ [1]

3. $T(n) = 7T(n/7) + n$

Master Theorem: $a = 7$, $b = 7$, $f(n) = n$

- $n^{\log_7 7} = n^1 = n$
- $f(n) = n = \Theta(n)$

Case 2: $T(n) = \Theta(n \log n)$ [1]

4. $T(n) = 9T(n/3) + n^2$

Master Theorem: $a = 9$, $b = 3$, $f(n) = n^2$

- $n^{\log_3 9} = n^2$
- $f(n) = n^2 = \Theta(n^2)$

Case 2: $T(n) = \Theta(n^2 \log n)$ [1]

5. $T(n) = 8T(n/2) + n^3$

Master Theorem: $a = 8$, $b = 2$, $f(n) = n^3$

- $n^{\log_2 8} = n^3$
- $f(n) = n^3 = \Theta(n^3)$

Case 2: $T(n) = \Theta(n^3 \log n)$ [1]

6. $T(n) = 49T(n/25) + n^{3/2} \log n$

Master Theorem: $a = 49$, $b = 25$, $f(n) = n^{3/2} \log n$

- $n^{\log_{25} 49} \approx n^{1.21}$
- $f(n) = n^{1.5} \log n = \Omega(n^{1.21+\epsilon})$ for $\epsilon < 0.29$

Check regularity: $49f(n/25) = 49 \cdot (n/25)^{1.5} \log(n/25) \leq c \cdot n^{1.5} \log n$ for $c < 1$

Case 3: $T(n) = \Theta(n^{3/2} \log n)$ [1]

7. $T(n) = T(n-1) + 2$

Unrolling:
$T(n) = T(n-1) + 2 = T(n-2) + 4 = \ldots = T(0) + 2n$

Answer: $T(n) = \Theta(n)$ [1]

8. $T(n) = T(n-1) + n^c$ where $c \geq 1$

Unrolling:
$T(n) = \sum_{i=1}^{n} i^c = \Theta(n^{c+1})$

Answer: $T(n) = \Theta(n^{c+1})$ [1]

9. $T(n) = T(n-1) + c^n$ where $c > 1$

Unrolling:
$T(n) = \sum_{i=1}^{n} c^i = c \cdot \frac{c^n - 1}{c-1} = \Theta(c^n)$

Answer: $T(n) = \Theta(c^n)$ [1]

10. $T(n) = 2T(n-1) + 1$

Unrolling:
$T(n) = 2T(n-1) + 1 = 2(2T(n-2) + 1) + 1 = 2^2 T(n-2) + 2 + 1$
$= 2^k T(n-k) + (2^{k-1} + \ldots + 2 + 1) = 2^n T(0) + 2^n - 1$

Answer: $T(n) = \Theta(2^n)$ [1]

11. $T(n) = T(n-1) + 1$

Answer: $T(n) = \Theta(n)$ [1]

Problem 4: FFT Computations

Part 1: FFT of (1, 0, 0, 0)

For $n = 4$, $\omega = e^{2\pi i/4} = i$ (primitive 4th root of unity)[1]

FFT:

$$\text{FFT}(1,0,0,0) = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & \omega & \omega^2 & \omega^3 \\ 1 & \omega^2 & \omega^4 & \omega^6 \\ 1 & \omega^3 & \omega^6 & \omega^9 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}$$

Inverse FFT: (1, 0, 0, 0) is the FFT of $(1/4, 1/4, 1/4, 1/4)$[1]

Part 2: FFT of (1, 0, 1, -1)

$$\text{FFT}(1,0,1,-1) = \begin{pmatrix} 1+1-1 \\ 1+0\cdot i-\omega^2+\omega^3 \\ 1+1+1 \\ 1-\omega^2-\omega^3 \end{pmatrix}$$

With $\omega = i$, $\omega^2 = -1$, $\omega^3 = -i$:

$$= \begin{pmatrix} 1 \\ 1-(-1)+(-i) = 2-i \\ 3 \\ 1+1+i = 2+i \end{pmatrix}$$

Inverse: Apply inverse FFT to get original sequence[1]

Problem 5: Matrix Multiplication from Squaring

Given: Algorithm that squares $n \times n$ matrix in $\Theta(n^\alpha)$ time[1]

Goal: Multiply two different matrices A and B in $\Theta(n^\alpha)$ time[1]

Algorithm:
Use the identity:
$(A+B)^2 = A^2 + AB + BA + B^2$
$AB + BA = (A+B)^2 - A^2 - B^2$

If A and B commute (which they don't generally), we'd have $AB = BA$, but we need a different approach.[1]

Better identity:

$$\begin{pmatrix} A & B \\ 0 & 0 \end{pmatrix}^2 = \begin{pmatrix} A^2 & AB \\ 0 & 0 \end{pmatrix}$$

But this requires a $2n \times 2n$ matrix, giving time $\Theta((2n)^\alpha) = \Theta(2^\alpha n^\alpha)$, which is not $\Theta(n^\alpha)$.[1]

Correct approach:
$(A+B)^2 - (A-B)^2 = 4AB$

So: $AB = \frac{1}{4}[(A+B)^2 - (A-B)^2]$

Algorithm:

1. Compute $C = A + B$ in $O(n^2)$ time
2. Compute $D = A - B$ in $O(n^2)$ time
3. Square C to get C^2 in $\Theta(n^\alpha)$ time
4. Square D to get D^2 in $\Theta(n^\alpha)$ time
5. Compute $AB = (C^2 - D^2)/4$ in $O(n^2)$ time

Total time: $\Theta(n^\alpha) + O(n^2) = \Theta(n^\alpha)$ since $\alpha \geq 2$ [1]

Problem 7: k-way Merge

Part (a): Sequential Merging

Algorithm:

```
Merge arrays A_1 and A_2 → Result_1 (size 2n)
Merge Result_1 and A_3 → Result_2 (size 3n)
...
Merge Result_{k-2} and A_k → Final (size kn)
```

Time complexity:

- Merge 1: $O(2n)$
- Merge 2: $O(3n)$
- ...
- Merge $k-1$: $O(kn)$

Total: $O(2n + 3n + \ldots + kn) = O(n(2 + 3 + \ldots + k)) = O(n \cdot k^2)$ [1]

Part (b): Divide-and-Conquer Approach

Algorithm:

```
k_way_merge(arrays, k):
   if k == 1:
      return arrays[^0]
   if k == 2:
      return merge(arrays[^0], arrays[^1])

   mid = k/2
   left = k_way_merge(arrays[0:mid], mid)
```

```
    right = k_way_merge(arrays[mid:k], k-mid)
    return merge(left, right)
```

Recurrence: $T(k, n) = 2T(k/2, n) + O(kn)$

The merge step combines two sorted arrays of size $kn/2$ each, taking $O(kn)$ time.[1]

Solving: $T(k, n) = O(kn \log k)$

Better approach using min-heap:

```
1. Create min-heap with first element from each array
2. Repeat kn times:
      Extract minimum from heap
      Add to result
      Insert next element from same array into heap
```

Time: $O(kn \log k)$ - kn extractions, each $O(\log k)$[1]

Problem 8: Strassen's Algorithm Pseudocode

```
def strassen(A, B, n):
    # Base case: 1x1 matrix
    if n == 1:
        return A[^0][^0] * B[^0][^0]

    # Divide matrices into quadrants
    mid = n // 2
    A11, A12, A21, A22 = partition(A, mid)
    B11, B12, B21, B22 = partition(B, mid)

    # Compute 7 products recursively
    P1 = strassen(A11 + A22, B11 + B22, mid)
    P2 = strassen(A21 + A22, B11, mid)
    P3 = strassen(A11, B12 - B22, mid)
    P4 = strassen(A22, B21 - B11, mid)
    P5 = strassen(A11 + A12, B22, mid)
    P6 = strassen(A21 - A11, B11 + B12, mid)
    P7 = strassen(A12 - A22, B21 + B22, mid)

    # Combine results
    C11 = P1 + P4 - P5 + P7
    C12 = P3 + P5
    C21 = P2 + P4
    C22 = P1 + P3 - P2 + P6

    return combine(C11, C12, C21, C22)
```

Time complexity: $T(n) = 7T(n/2) + O(n^2) = O(n^{\log_2 7}) \approx O(n^{2.81})$[1]

Problem 13: Majority Element

Part (a): $O(n \log n)$ Algorithm

Divide-and-Conquer:

```
find_majority(A, left, right):
   if left == right:
      return A[left]

   mid = (left + right) / 2
   left_maj = find_majority(A, left, mid)
   right_maj = find_majority(A, mid+1, right)

   if left_maj == right_maj:
      return left_maj

   # Count occurrences of both candidates
   left_count = count(A, left, right, left_maj)
   right_count = count(A, left, right, right_maj)

   if left_count > (right - left + 1) / 2:
      return left_maj
   if right_count > (right - left + 1) / 2:
      return right_maj

   return None
```

Correctness: If a majority element exists in the full array, it must be a majority in at least one half.[1]

Time complexity: $T(n) = 2T(n/2) + O(n) = O(n \log n)$[1]

Part (b): Linear-Time Algorithm (Boyer-Moore)

Algorithm:

```
find_majority_linear(A):
   # Phase 1: Find candidate
   candidate = None
   count = 0

   for i in range(n):
      if count == 0:
         candidate = A[i]
         count = 1
      elif A[i] == candidate:
         count += 1
      else:
         count -= 1

   # Phase 2: Verify candidate
   actual_count = count_occurrences(A, candidate)
   if actual_count > n/2:
```

```
      return candidate
   return None
```

Following the hint (pairing approach):

```
majority_linear(A):
   while len(A) > 1:
      new_A = []
      for i in range(0, len(A), 2):
         if i+1 < len(A):
            if A[i] == A[i+1]:
               new_A.append(A[i])
         else:
            new_A.append(A[i])
      A = new_A

   if len(A) == 1:
      candidate = A[^0]
      verify candidate in original array
```

Time: $O(n + n/2 + n/4 + \ldots) = O(n)$ [1]

Problem 16: Fixed Point in Sorted Array

Problem: Find index i where $A[i] = i$ in sorted distinct array [1]

Divide-and-Conquer (Binary Search variant):

```
find_fixed_point(A, left, right):
   if left > right:
      return -1  # No fixed point

   mid = (left + right) // 2

   if A[mid] == mid:
      return mid
   elif A[mid] > mid:
      # All elements to the right are also > their indices
      return find_fixed_point(A, left, mid - 1)
   else:  # A[mid] < mid
      # All elements to the left are also < their indices
      return find_fixed_point(A, mid + 1, right)
```

Why this works:

- If $A[mid] > mid$, then for all $i > mid$,
  $A[i] \geq A[mid] + (i - mid) > mid + (i - mid) = i$
- Similarly, if $A[mid] < mid$, all indices to the left can't be fixed points [1]

Time complexity: $T(n) = T(n/2) + O(1) = O(\log n)$ [1]

Problem 18: k-th Smallest in Two Sorted Lists

Algorithm:

```
def kth_smallest(A, B, k, a_start=0, a_end=None, b_start=0, b_end=None):
    if a_end is None:
        a_end = len(A) - 1
    if b_end is None:
        b_end = len(B) - 1

    # Base cases
    if a_start > a_end:
        return B[b_start + k]
    if b_start > b_end:
        return A[a_start + k]
    if k == 0:
        return min(A[a_start], B[b_start])

    # Binary search approach
    a_mid = (a_start + a_end) // 2
    b_mid = (b_start + b_end) // 2

    if a_mid - a_start + b_mid - b_start < k:
        # kth element is in the right half
        if A[a_mid] > B[b_mid]:
            return kth_smallest(A, B, k - (b_mid - b_start + 1),
                                a_start, a_end, b_mid + 1, b_end)
        else:
            return kth_smallest(A, B, k - (a_mid - a_start + 1),
                                a_mid + 1, a_end, b_start, b_end)
    else:
        # kth element is in the left half
        if A[a_mid] > B[b_mid]:
            return kth_smallest(A, B, k, a_start, a_mid - 1,
                                b_start, b_end)
        else:
            return kth_smallest(A, B, k, a_start, a_end,
                                b_start, b_mid - 1)
```

Time complexity: $O(\log m + \log n)$ [1]

Problem 21: Fibonacci Growth Bounds

Part (a): Prove $F_n \geq 2^{0.5n}$ for $n \geq 6$

Base cases:

- $F_6 = 8 \geq 2^3 = 8$ ✓
- $F_7 = 13 \geq 2^{3.5} \approx 11.3$ ✓ [1]

Inductive step: Assume $F_k \geq 2^{0.5k}$ for all $k < n$

$F_n = F_{n-1} + F_{n-2} \geq 2^{0.5(n-1)} + 2^{0.5(n-2)}$
$= 2^{0.5n} \cdot (2^{-0.5} + 2^{-1}) = 2^{0.5n} \cdot (\frac{1}{\sqrt{2}} + \frac{1}{2})$
$\approx 2^{0.5n} \cdot 1.207 > 2^{0.5n}$ ✓ [1]

Part (b): Find $c < 1$ such that $F_n \leq 2^{cn}$

Using golden ratio: $F_n = \frac{\phi^n - \psi^n}{\sqrt{5}}$ where $\phi = \frac{1+\sqrt{5}}{2} \approx 1.618$

$F_n \approx \frac{\phi^n}{\sqrt{5}} < \phi^n = \left(\frac{1+\sqrt{5}}{2}\right)^n$

Since $\phi \approx 1.618 < 2$, we can take $c = \log_2 \phi \approx 0.694$

Answer: $c = 0.694$ works[1]

Part (c): Largest c for $F_n = \Omega(2^{cn})$

Same analysis gives $c = \log_2 \phi \approx 0.694$[1]

This completes comprehensive solutions for the Divide-and-Conquer problem set! Each solution includes algorithms, correctness proofs, and complexity analysis as requested.[1]

⁂

1. AAD_QB_1_1-Divide-and-Conquer.pdf